


Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science

# CHAPTER 2

# Introduction to Deep Neural Networks

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

> *"The journey of a thousand models begins with a single neuron and a single backpropagation step."*
>
> — Pr. Abdelouahab Moussaoui

# Introduction to Deep Neural Networks


## Prof. Abdelouahab Moussaoui


## Academic Year: 2025-2026


**Abstract**

This comprehensive course provides a rigorous and practical introduction to **Deep Learning**, designed specifically for 4th year engineering students specializing in Artificial Intelligence. The document bridges the gap between theoretical foundations and industrial applications, offering a structured progression from fundamental concepts to state-of-the-art architectures.

The field of deep learning has undergone a remarkable transformation over the past decade, evolving from a niche academic discipline into the fundamental driver of the current artificial intelligence revolution. From computer vision systems that surpass human performance to large language models capable of engaging in sophisticated dialogue, deep learning has demonstrated an unprecedented capacity to solve complex problems across virtually every domain. Yet beneath the impressive results lies a sophisticated mathematical machinery that demands thorough understandingnot merely the ability to call high-level APIs, but genuine insight into the mechanisms of gradient-based learning, the dynamics of optimization, and the principles of generalization.

# Contents

**Preamble** **7**

**1 History and Evolution of Deep Learning** **10**
1.1 The Notion of Depth . . . 10
1.2 Hierarchical Processing Principle . . . 10
1.3 The Universal Approximation Theorem . . . 11
1.4 The Early Days (1943-1980) . . . 11
1.4.1 McCulloch-Pitts Neuron (1943) . . . 11
1.4.2 The Perceptron (1958) . . . 11
1.4.3 The XOR Problem and the First AI Winter . . . 11
1.5 The Emergence of Convolutional Networks . . . 11
1.5.1 Neocognitron (1980) . . . 11
1.5.2 LeNet-5 (1998) . . . 11
1.6 The ImageNet Revolution (2012) . . . 12
1.6.1 AlexNet: The Breakthrough . . . 12
1.6.2 Evolution of CNN Architectures . . . 12
1.7 The Attention Era and Transformers . . . 13
1.7.1 "Attention is All You Need" (2017) . . . 13
1.8 Large Language Models (LLMs) . . . 13
1.8.1 BERT: Bidirectional Understanding . . . 13
1.8.2 The GPT Family . . . 13
1.8.3 Key Innovations in LLMs . . . 13
1.9 Section Summary . . . 14

**2 Backpropagation: Mathematical Foundations** **15**
2.1 Mathematical Prerequisites . . . 15
2.2 The Multi-Layer Perceptron (MLP) . . . 15
2.3 Forward Propagation . . . 16
2.4 Backpropagation Derivation . . . 16
2.5 Gradient Computation . . . 16
2.6 Complete Backpropagation Algorithm . . . 17
2.7 Detailed Example: 2-Layer Network with Sigmoid . . . 17
2.8 Vanishing and Exploding Gradients . . . 18
2.9 Solution Techniques . . . 18
2.10 Computational Graphs and Automatic Differentiation . . . 18
2.11 Section Summary . . . 19

**3 Activation Functions** **20**
3.1 Role of Activation Functions . . . 20
3.2 Desirable Properties . . . 20
3.3 The 10 Most Important Activation Functions . . . 20
3.3.1 1. Sigmoid Function . . . 20
3.3.2 2. Tanh Function . . . 21
3.3.3 3. ReLU (Rectified Linear Unit) . . . 21

3.3.4 4. Leaky ReLU . . . 22
3.3.5 5. ELU (Exponential Linear Unit) . . . 22
3.3.6 6. SELU (Scaled Exponential Linear Unit) . . . 23
3.3.7 7. GELU (Gaussian Error Linear Unit) . . . 23
3.3.8 8. Softplus . . . 24
3.3.9 9. Swish / SiLU (Sigmoid Linear Unit) . . . 24
3.3.10 10. Softmax (for Multi-class Classification) . . . 25
3.4 Comparison and Selection Guide . . . 25
3.5 Practical Recommendations . . . 25

**4 Loss Functions 26**
4.1 Role of Loss Functions . . . 26
4.2 Loss Functions for Regression . . . 26
4.2.1 Mean Squared Error (MSE) . . . 26
4.2.2 Mean Absolute Error (MAE) . . . 26
4.2.3 Huber Loss . . . 26
4.3 Loss Functions for Binary Classification . . . 27
4.3.1 Binary Cross-Entropy (Log Loss) . . . 27
4.3.2 Hinge Loss . . . 27
4.4 Loss Functions for Multi-class Classification . . . 27
4.4.1 Categorical Cross-Entropy . . . 27
4.4.2 Sparse Categorical Cross-Entropy . . . 27
4.4.3 Kullback-Leibler Divergence . . . 28
4.5 Loss Functions Comparison . . . 28

**5 Optimizers 29**
5.1 Gradient Descent Variants . . . 29
5.1.1 Batch Gradient Descent . . . 29
5.1.2 Stochastic Gradient Descent (SGD) . . . 29
5.1.3 Mini-batch Gradient Descent . . . 29
5.2 Optimizers with Momentum . . . 29
5.2.1 SGD with Momentum . . . 29
5.2.2 Nesterov Accelerated Gradient (NAG) . . . 30
5.3 Adaptive Optimizers . . . 30
5.3.1 AdaGrad (Adaptive Gradient) . . . 30
5.3.2 RMSprop (Root Mean Square Propagation) . . . 30
5.3.3 Adam (Adaptive Moment Estimation) . . . 30
5.3.4 AdamW (Adam with Weight Decay) . . . 31
5.3.5 Nadam (Nesterov Adam) . . . 31
5.4 Optimizers Comparison . . . 31
5.5 Practical Recommendations . . . 31
5.6 Learning Rate Selection Guide . . . 32
5.7 Learning Rate Scheduling Strategies . . . 32

**6 Batch Size and Epochs** **33**
6.1 Introduction to Hyperparameters . . . 33
6.2 Batch Size . . . 33
6.2.1 Mathematical Foundation . . . 33
6.2.2 Types of Batch Sizes . . . 34
6.2.3 Impact of Batch Size on Training . . . 34
6.2.4 Batch Size Categories . . . 34
6.2.5 The Linear Scaling Rule . . . 35
6.2.6 Gradient Accumulation . . . 35
6.3 Epochs . . . 35
6.3.1 Mathematical Relationships . . . 36
6.3.2 Impact of Number of Epochs . . . 36
6.3.3 Determining Optimal Epochs . . . 36
6.3.4 Epoch Guidelines by Architecture . . . 36
6.4 Relationship Between Batch Size and Epochs . . . 37
6.5 Practical Recommendations . . . 37

**7 Validation Techniques** **38**
7.1 Fundamental Concepts . . . 38
7.2 Bias-Variance Tradeoff . . . 38
7.3 Data Splitting . . . 38
7.3.1 Train/Validation/Test Split . . . 38
7.4 Cross-Validation Techniques . . . 38
7.4.1 k-Fold Cross-Validation . . . 38
7.4.2 Stratified k-Fold . . . 39
7.4.3 Leave-One-Out Cross-Validation (LOOCV) . . . 39
7.5 Learning Curves . . . 39
7.6 Performance Metrics . . . 39
7.6.1 Classification Metrics . . . 39
7.6.2 Regression Metrics . . . 40
7.7 Early Stopping Techniques . . . 40
7.7.1 Fixed Epoch Stopping . . . 40
7.7.2 Naive Best Solution Stopping . . . 40
7.7.3 Early Stopping with Patience . . . 40
7.7.4 Early Stopping with Improvement Threshold . . . 40
7.8 Section Summary . . . 41

**8 First TensorFlow Example** **42**
8.1 Introduction to TensorFlow/Keras . . . 42
8.2 MNIST Classification Example . . . 42
8.2.1 Data Loading and Preprocessing . . . 42
8.2.2 Model Architecture . . . 42
8.2.3 Model Compilation . . . 43
8.2.4 Model Training . . . 43
8.2.5 Model Evaluation . . . 43
8.3 Understanding Dense Layers . . . 43
8.4 Advanced Model Configurations . . . 44

8.4.1 Adding Regularization . . . 44
8.4.2 Early Stopping Callback . . . 44
8.5 Model Saving and Loading . . . 45
8.6 Practical Recommendations . . . 45

**9 Regularization Techniques 46**
9.1 Why Regularization? . . . 46
9.2 The 8 Most Important Regularization Techniques . . . 46
9.2.1 1. L2 Regularization (Weight Decay) . . . 46
9.2.2 2. L1 Regularization (Lasso) . . . 46
9.2.3 3. Dropout . . . 47
9.2.4 4. Batch Normalization . . . 47
9.2.5 5. Early Stopping . . . 47
9.2.6 6. Data Augmentation . . . 48
9.2.7 7. Label Smoothing . . . 48
9.2.8 8. Gradient Clipping . . . 49
9.3 Regularization Comparison . . . 49
9.4 Practical Recommendations . . . 49

**10 Model Saving and Deployment 51**
10.1 Why Model Persistence Matters . . . 51
10.2 Saving and Loading Models . . . 51
10.2.1 What Gets Saved? . . . 51
10.2.2 Saving Weights Only . . . 51
10.2.3 Saving Complete Model (H5 Format) . . . 51
10.2.4 SavedModel Format (Recommended) . . . 52
10.2.5 Checkpoints During Training . . . 52
10.3 Model Conversion and Optimization . . . 52
10.3.1 TensorFlow Lite for Mobile/Edge . . . 52
10.3.2 TensorFlow.js for Web Deployment . . . 52
10.3.3 ONNX for Framework Interoperability . . . 53
10.4 Production Deployment . . . 53
10.4.1 TensorFlow Serving . . . 53
10.4.2 REST API with Flask . . . 53
10.4.3 FastAPI Alternative . . . 53
10.5 Model Versioning and Monitoring . . . 54
10.6 Practical Recommendations . . . 54

**General Conclusion 55**

# Preamble

## About this Document

This document constitutes the complete course material for **Deep Learning** instruction designed for **4th Year** engineering students, specializing in **Artificial Intelligence**. It has been carefully crafted to provide a progressive and in-depth understanding of fundamental concepts, advanced techniques, and practical applications of deep neural networks.

The course is structured into **thematic sections** covering the entire spectrum of deep learning, from mathematical foundations to the most recent architectures such as Transformers and Large Language Models (LLMs). Each section has been developed with constant attention to mathematical rigor and pedagogical clarity, integrating concrete examples, visualizations, and practical implementations.

## Target Audience

This course is primarily intended for:

- **4th Year Engineering Students**, Artificial Intelligence specialization
- **Researchers and practitioners** wishing to strengthen their deep learning foundations
- **Developers** seeking to understand the internal mechanisms of the models they use

The prerequisites for following this course are:

- Solid knowledge of **Python programming**
- Basic notions of **differential calculus and linear algebra**
- Familiarity with fundamental **machine learning** concepts

**Prerequisites:** While the course is self-contained, students will benefit from prior exposure to:

- Matrix operations and vector calculus
- Probability and statistics fundamentals
- Basic optimization concepts
- Previous experience with scikit-learn or similar ML libraries

**How to Use This Document:**

- **Sequential Study**: Follow the sections in order, as concepts build progressively
- **Hands-on Practice**: Implement all code examples and complete the exercises
- **Reference Use**: Consult specific sections as needed during projects
- **Experimentation**: Modify parameters, architectures, and datasets to deepen understanding

## Course Structure

The document is organized into several sections, each addressing a fundamental aspect of deep learning:

1. **Section 1: History and Evolution of Deep Learning** - Historical context and motivation
2. **Section 2: Backpropagation** - Mathematical foundations of learning
3. **Section 3: Activation Functions** - Essential non-linearities
4. **Section 4: Loss Functions** - Optimization objectives
5. **Section 5: Optimizers** - Parameter update algorithms
6. **Section 6: Batch Size and Epochs** - Fundamental hyperparameters
7. **Section 7: Validation Techniques** - Evaluation and generalization
8. **Section 8: First TensorFlow Example** - Hands-on practice
9. **Section 9: Regularization** - Preventing overfitting
10. **Section 10: Model Saving and Deployment** - Moving to production

## Learning Objectives

### General Objective

The main objective of this course is to train students in the **design, implementation, and deployment of robust and performant deep learning models**, providing both the theoretical foundations and practical skills necessary to tackle current challenges in artificial intelligence.

### Targeted Skills

This course aims to develop the following skills, aligned with industry and research needs:

**Table 1: Skills Developed**

| Skill | Description |
|---|---|
| Analysis | Ability to analyze a problem and choose the appropriate architecture |
| Implementation | Mastery of modern frameworks (TensorFlow/Keras) |
| Optimization | Systematic hyperparameter tuning |
| Evaluation | Use of appropriate metrics and rigorous validation |
| Deployment | Production deployment of models via APIs and containers |
| Research | Critical reading of literature and technological watch |

## How to Use This Course

To get the most out of this course, we recommend the following approach:

1. **Sequential reading**: Follow the section order, as concepts are built progressively
2. **Practical experimentation**: Implement the examples and complete the exercises
3. **Systematic visualization**: Plot learning curves for each experiment
4. **Iterative tuning**: Start with recommended default values, then adjust
5. **Documentation**: Keep an experiment log to capitalize on learnings

**Remarque 0.1**

This course is designed to be both a pedagogical support and a practical reference. Feel free to consult it regularly during your projects to review fundamental concepts and best practices.

*The journey of a thousand models begins with*
*a simple neuron and a single backpropagation step.*

*From one neuron, one gradient, one step*
*a thousand models arise.*

*One neuron listens, one gradient guides,*
*and a thousand models learn to see.*

*Prof. Abdelouahab Moussaoui*
*U.F.A.Setif 1, February 27, 2026*

# 1 History and Evolution of Deep Learning

## 1.1 The Notion of Depth

**Définition 1.1 : Network Depth**

The depth of a neural network refers to the number of successive layers that transform the input into the output. We distinguish:

- **Shallow networks**: 1 to 3 hidden layers
- **Deep networks**: More than 3 layers, sometimes hundreds

### Historical Perspective

In the 1980s and 1990s, hardware constraints severely limited network size. A network with 3 hidden layers was considered "deep" due to:

- Computational constraints (processors could not efficiently train larger networks)
- Vanishing gradient (backpropagation suffered beyond a few layers)
- Limited data (small datasets did not justify deep architectures)

### Modern Perspective

Today, the notion of depth has considerably evolved:

- **ResNet-152**: 152 layers (Microsoft Research, 2015)
- **GPT-3**: 96 Transformer layers, 175 billion parameters
- **Google's Switch Transformer**: Over one trillion parameters

## 1.2 Hierarchical Processing Principle

Beyond layer count, the essential principle is hierarchical processing. Each successive layer learns increasingly abstract representations:

1. **Layer 1**: Detects simple features (edges, textures, color blobs)
2. **Layer 2**: Combines simple features into patterns (corners, circles, stripes)
3. **Layer 3**: Forms object parts (eyes, wheels, windows)
4. **Layer 4**: Recognizes complete objects (faces, cars, houses)
5. **Layer 5+**: Understands abstract concepts (scenes, actions, emotions)

## 1.3 The Universal Approximation Theorem

**Théorème 1.1 : Cybenko, 1989 - Universal Approximation**

A feedforward neural network with a single hidden layer containing a finite number of neurons and a sigmoid activation function can approximate any continuous function on a compact subset of $\mathbb{R}^n$, to any desired degree of accuracy.

**Propriété 1.1 : Efficiency of Deep Networks**

There exist functions that can be represented compactly by a deep network of size $O(n)$ but would require a shallow network of size $O(2^n)$ to achieve the same approximation accuracy.

## 1.4 The Early Days (1943-1980)

### 1.4.1 McCulloch-Pitts Neuron (1943)

The first mathematical model of a biological neuron:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

where $x_i$ are binary inputs, $w_i \in \{0, 1\}$ are fixed weights, and $\theta$ is a threshold.

### 1.4.2 The Perceptron (1958)

Frank Rosenblatt introduced the first algorithm capable of learning weights from data:

$$w_i^{(t+1)} = w_i^{(t)} + \eta(y - \hat{y})x_i$$

### 1.4.3 The XOR Problem and the First AI Winter

Minsky and Papert's 1969 book "Perceptrons" demonstrated that single-layer perceptrons cannot solve the XOR problem. This led to the first "AI Winter" (1970s-1980s), with reduced funding and interest in neural networks.

## 1.5 The Emergence of Convolutional Networks

### 1.5.1 Neocognitron (1980)

Kunihiko Fukushima proposed the Neocognitron, inspired by Hubel and Wiesel's work on the visual cortex.

### 1.5.2 LeNet-5 (1998)

Yann LeCun revolutionized handwritten digit recognition with LeNet-5:

**Table 2: LeNet-5 Architecture**

| Layer | Output Size | Parameters |
|---|---|---|
| Input | $32 \times 32$ | - |
| Conv1 (6 filters $5 \times 5$) | $28 \times 28 \times 6$ | 156 |
| Pooling | $14 \times 14 \times 6$ | - |
| Conv2 (16 filters $5 \times 5$) | $10 \times 10 \times 16$ | 2,416 |
| Pooling | $5 \times 5 \times 16$ | - |
| Conv3 (120 filters $5 \times 5$) | $1 \times 1 \times 120$ | 48,120 |
| FC1 (84 neurons) | 84 | 10,164 |
| Output (10 neurons) | 10 | 850 |

## 1.6 The ImageNet Revolution (2012)

### 1.6.1 AlexNet: The Breakthrough

In 2012, Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton achieved a dramatic breakthrough in the ImageNet competition with:

- **ReLU activations**: Solved vanishing gradient
- **Dropout**: Prevented overfitting
- **GPU training**: 100x speedup
- **Data augmentation**: 2048x more training examples

### 1.6.2 Evolution of CNN Architectures

**Table 3: Major CNN Architectures Evolution (1998-2024)**

| Year | Architecture | Layers | Key Innovation |
|---|---|---|---|
| 1998 | LeNet-5 | 7 | First modern CNN |
| 2012 | AlexNet | 8 | ReLU, Dropout, GPU training |
| 2014 | VGGNet | 19 | Simplicity, uniform depth |
| 2014 | GoogLeNet | 22 | Inception modules |
| 2015 | ResNet | 152 | Skip connections |
| 2016 | ResNeXt | 101 | Aggregated transformations |
| 2017 | DenseNet | 264 | Dense connections |
| 2017 | Xception | 126 | Depthwise separable conv |
| 2017 | MobileNet | 53 | Lightweight mobile CNN |
| 2018 | SENet | 154 | Squeeze-Excitation blocks |
| 2019 | EfficientNet | 813 | NAS + compound scaling |
| 2021 | ConvNeXt | 18-36 | Modernized ResNet |
| 2023 | MaxViT | 31-58 | Hybrid CNN-Transformer |

## 1.7 The Attention Era and Transformers

**Définition 1.2 : Attention**

Attention allows a model to focus on relevant parts of the input when producing each output element:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q$ (queries), $K$ (keys), and $V$ (values) are derived from the input.

### 1.7.1 "Attention is All You Need" (2017)

The Transformer architecture, introduced by Vaswani et al., revolutionized NLP and beyond with:

- Multi-head self-attention
- Positional encodings
- Feed-forward networks
- Residual connections and layer normalization

## 1.8 Large Language Models (LLMs)

### 1.8.1 BERT: Bidirectional Understanding

BERT (Bidirectional Encoder Representations from Transformers) introduced masked language modeling and next-sentence prediction.

### 1.8.2 The GPT Family

**Table 4: Evolution of GPT Models**

| Model | Year | Parameters | Training Data |
|---|---|---|---|
| GPT-1 | 2018 | 117M | BooksCorpus |
| GPT-2 | 2019 | 1.5B | WebText |
| GPT-3 | 2020 | 175B | Common Crawl |
| GPT-4 | 2023 | ∼1.8T | Multimodal |
| GPT-5 | 2025 | ∼10-50T | Multimodal + Real-time data |

### 1.8.3 Key Innovations in LLMs

- **In-context learning**: Learning from examples without gradient updates
- **Chain-of-thought reasoning**: Step-by-step reasoning

- **Reinforcement Learning from Human Feedback (RLHF)**: Alignment with human preferences
- **Multimodal capabilities**: Processing text, images, audio

## 1.9 Section Summary

1. Deep learning's success comes from hierarchical feature learning
2. The field has evolved through multiple waves since 1943
3. Key architectures: LeNet AlexNet ResNet Transformers LLMs
4. Each breakthrough addressed specific limitations of previous approaches

# 2 Backpropagation: Mathematical Foundations

## 2.1 Mathematical Prerequisites

**Définition 2.1 : Gradient**

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is a vector of partial derivatives:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \cdots & \frac{\partial f}{\partial x_n} \end{bmatrix}^T$$

**Définition 2.2 : Jacobian Matrix**

For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is an $m \times n$ matrix:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

**Théorème 2.1 : Chain Rule**

For composite functions:

- Scalar case: $\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$
- Vector case: $\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \cdot \frac{\partial \mathbf{g}}{\partial \mathbf{x}}$

## 2.2 The Multi-Layer Perceptron (MLP)

Consider an MLP with $L$ layers. Let:

- $\mathbf{x} \in \mathbb{R}^{d_0}$: input vector
- $\mathbf{y} \in \mathbb{R}^{d_L}$: target output
- $\hat{\mathbf{y}} \in \mathbb{R}^{d_L}$: predicted output
- $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$: weight matrix for layer $l$
- $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$: bias vector for layer $l$
- $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l}$: pre-activation at layer $l$
- $\mathbf{a}^{(l)} \in \mathbb{R}^{d_l}$: activation at layer $l$
- $f^{(l)}$: activation function for layer $l$

## 2.3 Forward Propagation

Forward propagation computes the output given an input:

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x}\\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\\ \mathbf{a}^{(l)} &= f^{(l)}(\mathbf{z}^{(l)})\\ \hat{\mathbf{y}} &= \mathbf{a}^{(L)}\\ \mathcal{L} &= \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})\end{aligned}$$

## 2.4 Backpropagation Derivation

Our goal is to compute the gradients of the loss function with respect to all parameters:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \quad \forall l$$

**Définition 2.3 : Error Signal**

The error signal at layer $l$ is defined as:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$$

This represents how much the pre-activations at layer $l$ affect the final loss.

**Théorème 2.2 : Backpropagation Equations**

The error signals satisfy the following recurrence relation:

$$\boldsymbol{\delta}^{(l)} = \left((\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}\right) \odot f'(\mathbf{z}^{(l)})$$

where $\odot$ denotes element-wise multiplication.

## 2.5 Gradient Computation

For the biases:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

For the weights:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T$$

## 2.6 Complete Backpropagation Algorithm

1. **Forward pass**: Compute all activations

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x}\\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\\ \mathbf{a}^{(l)} &= f^{(l)}(\mathbf{z}^{(l)})\\ \hat{\mathbf{y}} &= \mathbf{a}^{(L)}\\ \mathcal{L} &= \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})\end{aligned}$$

2. **Backward pass**: Compute error signals

$$\begin{aligned}\boldsymbol{\delta}^{(L)} &= \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \odot f'(\mathbf{z}^{(L)})\\ \boldsymbol{\delta}^{(l)} &= ((\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}) \odot f'(\mathbf{z}^{(l)}) \quad \text{for } l = L-1 \text{ down to } 1\end{aligned}$$

3. **Gradient computation**

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} &= \boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T\\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} &= \boldsymbol{\delta}^{(l)}\end{aligned}$$

## 2.7 Detailed Example: 2-Layer Network with Sigmoid

Consider a network with:

- Input dimension: 2
- Hidden layer: 3 neurons, sigmoid activation
- Output layer: 1 neuron, sigmoid activation
- Loss: Binary cross-entropy

### Step 1: Forward Propagation

Given $\mathbf{x} = [0.5, 0.8]^T$, $\mathbf{y} = 1$:

$$\begin{aligned}\mathbf{z}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = [0.31, 0.57, 0.83]^T\\ \mathbf{a}^{(1)} &= \sigma(\mathbf{z}^{(1)}) = [0.577, 0.639, 0.696]^T\\ z^{(2)} &= \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)} = 0.6855\\ \hat{y} &= \sigma(z^{(2)}) = 0.664\end{aligned}$$

### Step 2: Loss Computation

$$\mathcal{L} = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})] = -\log(0.664) \approx 0.409$$

### Step 3: Backward Pass

$$\delta^{(2)} = \hat{y} - y = -0.336$$
$$\delta^{(1)} = [-0.0164, -0.0233, -0.0285]^T$$

### Step 4: Gradient Computation

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = [-0.194, -0.215, -0.234], \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.336$$

## 2.8 Vanishing and Exploding Gradients

**Définition 2.4 : Vanishing Gradient**

When gradients become extremely small as they propagate backward through many layers, preventing early layers from learning effectively.

**Causes:**

- Sigmoid/tanh activations saturate
- Repeated multiplication by small weights
- Deep architectures amplify the problem

**Définition 2.5 : Exploding Gradient**

When gradients become extremely large, causing unstable updates and numerical overflow.

## 2.9 Solution Techniques

**Table 5: Techniques to Address Gradient Problems**

| Technique | Problem Solved | How It Works |
|---|---|---|
| ReLU activation | Vanishing | Derivative = 1 for positive inputs |
| Skip connections | Vanishing | Gradient shortcuts |
| Gradient clipping | Exploding | Cap gradient norm |
| Batch normalization | Both | Stabilizes distributions |
| Careful initialization | Both | Proper weight scaling |

## 2.10 Computational Graphs and Automatic Differentiation

Modern deep learning frameworks use computational graphs for automatic differentiation. Two modes exist:

- **Forward mode**: Propagate derivatives from inputs to outputs
- **Reverse mode (backpropagation)**: Propagate from outputs to inputs

## 2.11 Section Summary

1. Backpropagation efficiently computes gradients using the chain rule
2. The algorithm has two passes: forward (compute output) and backward (compute gradients)
3. Error signals propagate backward through the network
4. Gradient vanishing/exploding are fundamental challenges in deep learning
5. Modern frameworks implement automatic differentiation based on these principles

# 3 Activation Functions

## 3.1 Role of Activation Functions

Without non-linear activations, a neural network would be equivalent to a linear transformation:

$$\mathbf{a}^{(L)} = \mathbf{W}^{(L)}(\cdots(\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)})\cdots) + \mathbf{b}^{(L)}$$

**Proposition 3.1**

A multi-layer network with only linear activations is equivalent to a single-layer linear network.

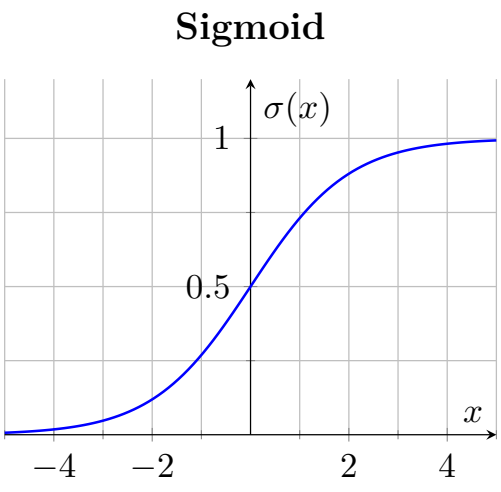
## 3.2 Desirable Properties

**Propriété 3.1 : Desirable Properties of Activation Functions**

- **Non-linear**: Enables learning complex functions
- **Differentiable**: Required for gradient-based learning
- **Computationally efficient**: Fast forward and backward passes
- **Non-saturating**: Avoids vanishing gradients
- **Zero-centered**: Helps convergence

## 3.3 The 10 Most Important Activation Functions

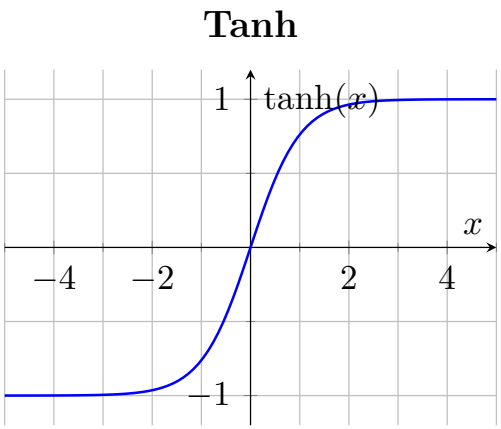
### 3.3.1 1. Sigmoid Function




$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- Output range: $(0, 1)$
- Differentiable everywhere
- Saturates for large $|x|$
- Not zero-centered
- **Use cases**: Binary classification output layer, gates in RNNs/LSTMs

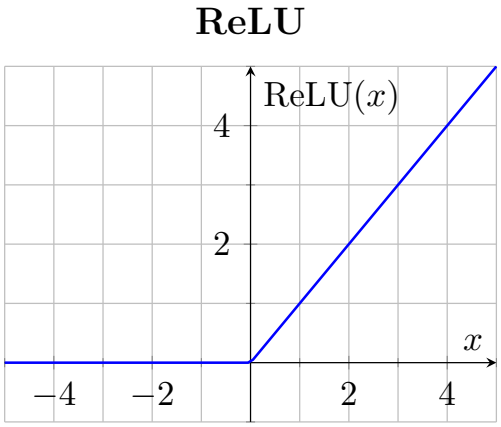
### 3.3.2 2. Tanh Function




$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$
$$\tanh'(x) = 1 - \tanh^2(x)$$

- Output range: $(-1, 1)$
- Zero-centered (unlike sigmoid)
- Saturates for large $|x|$
- **Use cases**: Hidden layers (historically), RNN cells

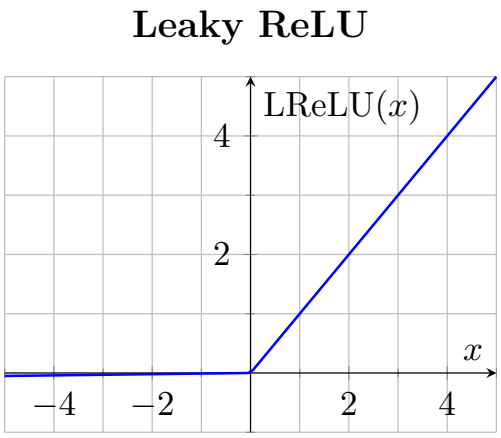
### 3.3.3 3. ReLU (Rectified Linear Unit)




$$\text{ReLU}(x) = \max(0, x)$$
$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

- Output range: $[0, \infty)$
- Computationally efficient
- No saturation for $x > 0$
- Dying ReLU problem for $x < 0$
- **Use cases**: Default choice for hidden layers, CNNs, MLPs

### 3.3.4 4. Leaky ReLU

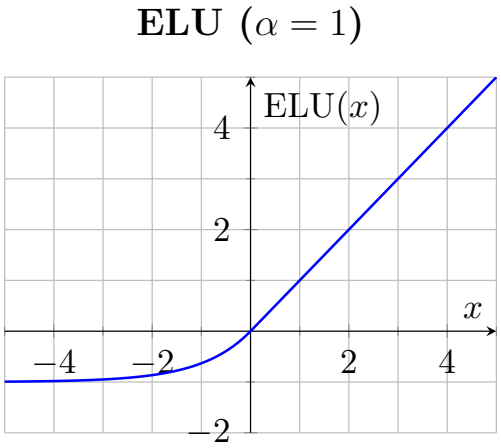



$$\text{LReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}, \alpha = 0.01$$

$$\text{LReLU}'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$

- Fixes dying ReLU
- Small negative slope
- **Use cases**: When ReLU causes dead neurons, GANs

### 3.3.5 5. ELU (Exponential Linear Unit)

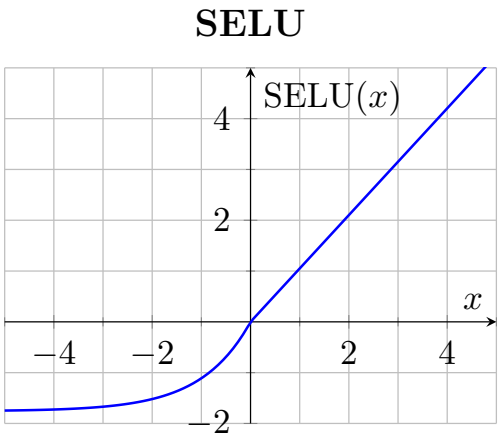



$$\text{ELU}(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

$$\text{ELU}'(x) = \begin{cases} 1 & x > 0 \\ \text{ELU}(x) + \alpha & x \leq 0 \end{cases}$$

- Smooth everywhere
- Negative values push mean toward zero
- More expensive than ReLU
- **Use cases**: Deep networks

### 3.3.6 6. SELU (Scaled Exponential Linear Unit)

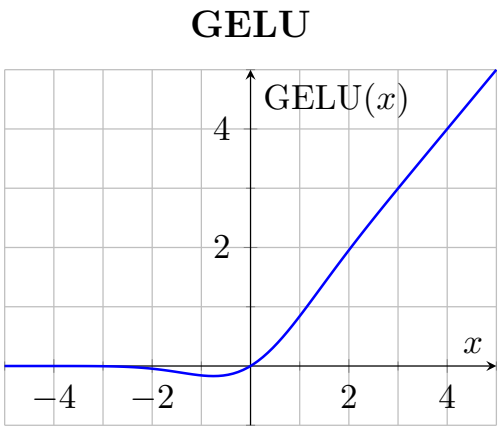



$$\text{SELU}(x) = \lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

$$\lambda \approx 1.0507, \alpha \approx 1.6733$$

- Self-normalizing
- Automatically normalizes activations
- Requires specific initialization
- **Use cases**: MLPs with proper initialization

### 3.3.7 7. GELU (Gaussian Error Linear Unit)




$$\text{GELU}(x) = x\Phi(x)$$

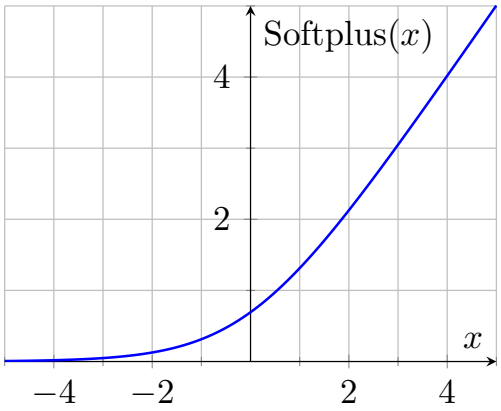
$$\Phi(x) = \int_{-\infty}^{x} \frac{e^{-t^2/2}}{\sqrt{2\pi}} dt$$

Approx: $0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$

- Smooth, non-monotonic
- Used in BERT, GPT
- Better performance on some tasks
- **Use cases**: Transformers, very deep networks

### 3.3.8 8. Softplus

**Softplus**

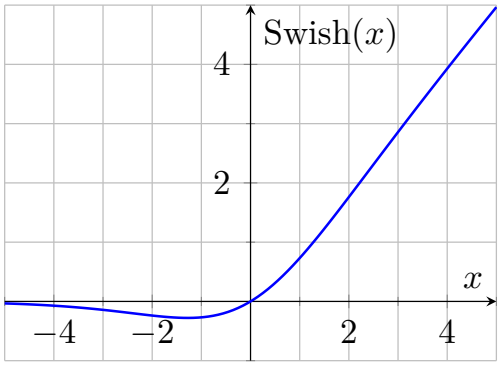



$$\text{Softplus}(x) = \log(1 + e^x)$$
$$\text{Softplus}'(x) = \sigma(x)$$

- Smooth approximation of ReLU
- Always positive
- More expensive than ReLU

### 3.3.9 9. Swish / SiLU (Sigmoid Linear Unit)

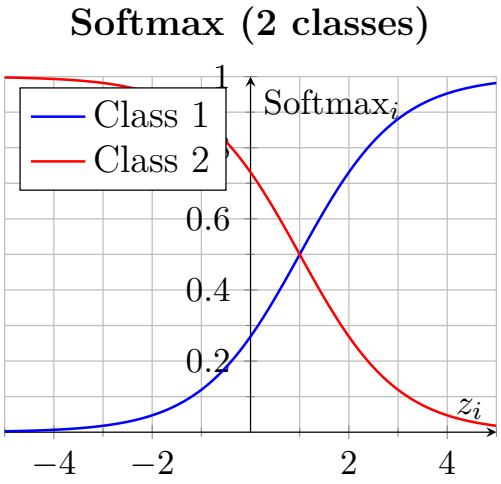
**Swish**




$$\text{Swish}(x) = x \cdot \sigma(\beta x)$$
$$\text{SiLU}(x) = x \cdot \sigma(x)$$
$$\text{SiLU}'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x))$$

- Smooth, non-monotonic
- No upper bound, lower bound
- Often outperforms ReLU
- **Use cases**: Deep networks, modern architectures

### 3.3.10 10. Softmax (for Multi-class Classification)




$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

$$\frac{\partial \text{softmax}_i}{\partial z_j} = \text{softmax}_i(\delta_{ij} - \text{softmax}_j)$$

- Outputs probability distribution
- All outputs sum to 1
- **Use cases**: Output layer for multi-class classification

## 3.4 Comparison and Selection Guide

**Table 6: Activation Functions Comparison**

| Function | Range | Pros | Best For |
|---|---|---|---|
| ReLU | $[0, \infty)$ | Fast, no saturation | Hidden layers (default) |
| Leaky ReLU | $(-\infty, \infty)$ | Fixes dying ReLU | GANs |
| ELU | $(-\infty, \infty)$ | Smooth, zero-mean | Deep networks |
| SELU | $(-\infty, \infty)$ | Self-normalizing | MLPs |
| GELU | $(-\infty, \infty)$ | Smooth, performance | Transformers |
| Swish | $(-\infty, \infty)$ | Smooth, performance | Deep networks |
| Sigmoid | $(0, 1)$ | Probabilistic | Binary output |
| Tanh | $(-1, 1)$ | Zero-centered | RNNs |
| Softmax | $(0, 1)^K$ | Probabilistic | Multi-class output |

## 3.5 Practical Recommendations

1. **Start with ReLU** for hidden layers in most cases
2. **Use Leaky ReLU or ELU** if you encounter dead neurons
3. **Use sigmoid/tanh** only for specific gates or historical reasons
4. **Use softmax** for multi-class classification output
5. **Consider GELU/Swish** for very deep networks or Transformers
6. **Use SELU** only with proper initialization (LeCun normal)

# 4 Loss Functions

## 4.1 Role of Loss Functions

**Définition 4.1 : Loss Function**

A loss function $\mathcal{L}(\hat{y}, y)$ measures the discrepancy between the predicted output $\hat{y}$ and the true target $y$. The goal of training is to minimize the average loss over the training dataset.

## 4.2 Loss Functions for Regression

### 4.2.1 Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Always non-negative
- Differentiable everywhere
- Penalizes large errors more heavily (quadratic)
- Sensitive to outliers

### 4.2.2 Mean Absolute Error (MAE)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Robust to outliers
- Not differentiable at 0
- Linear penalty for all errors

### 4.2.3 Huber Loss

$$\mathcal{L}_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

- Combines MSE and MAE
- Quadratic for small errors, linear for large errors
- Smooth and differentiable everywhere
- Parameter $\delta$ controls transition point

## 4.3 Loss Functions for Binary Classification

### 4.3.1 Binary Cross-Entropy (Log Loss)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

- Based on maximum likelihood estimation
- Penalizes confident wrong predictions heavily
- Requires $\hat{y}_i \in (0,1)$ (use sigmoid output)

### 4.3.2 Hinge Loss

$$\mathcal{L}_{\text{Hinge}} = \frac{1}{n}\sum_{i=1}^{n}\max(0, 1-y_i\hat{y}_i)$$

where $y_i \in \{-1, +1\}$.

- Used in SVMs
- Encourages margin maximization
- Not differentiable at the hinge point

## 4.4 Loss Functions for Multi-class Classification

### 4.4.1 Categorical Cross-Entropy

$$\mathcal{L}_{\text{CCE}} = -\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

where $y_{i,c}$ is 1 if sample $i$ belongs to class $c$, 0 otherwise.

- For one-hot encoded labels
- Works with softmax activation
- Penalizes incorrect class probabilities

### 4.4.2 Sparse Categorical Cross-Entropy

Same as categorical cross-entropy but accepts integer labels instead of one-hot encoded vectors.

### 4.4.3 Kullback-Leibler Divergence

$$\mathcal{L}_{\text{KL}}(P||Q) = \sum_x P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

- Measures difference between two probability distributions
- Always non-negative
- Not symmetric: $KL(P||Q) \neq KL(Q||P)$
- Used in variational inference

## 4.5 Loss Functions Comparison

**Table 7: Loss Functions Summary**

| Loss Function | Task | Output Activation | Properties |
|---|---|---|---|
| MSE | Regression | Linear | Sensitive to outliers |
| MAE | Regression | Linear | Robust to outliers |
| Huber | Regression | Linear | Best of both worlds |
| Binary Cross-Entropy | Binary Classification | Sigmoid | Probabilistic interpretatio |
| Categorical Cross-Entropy | Multi-class | Softmax | For one-hot labels |
| Sparse CCE | Multi-class | Softmax | For integer labels |
| Hinge | Binary Classification | Linear or tanh | Margin-based |

# 5 Optimizers

## 5.1 Gradient Descent Variants

### 5.1.1 Batch Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

- Uses gradient computed on **entire training dataset**
- Computational cost: High, Gradient variance: Low
- Not suitable for online learning

### 5.1.2 Stochastic Gradient Descent (SGD)

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t; x_i, y_i)$$

- Uses gradient computed on a **single random training example**
- Computational cost: Low, Gradient variance: High
- Suitable for online learning

### 5.1.3 Mini-batch Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t; x_{i:i+n}, y_{i:i+n})$$

- Uses gradient computed on a **small batch of $n$ examples**
- Most common approach in practice
- Typical batch sizes: 32, 64, 128, 256

## 5.2 Optimizers with Momentum

### 5.2.1 SGD with Momentum

$$v_t = \beta v_{t-1} + \eta \nabla_\theta \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - v_t$$

- Accumulates velocity in consistent directions
- $\beta$ (momentum coefficient) typically 0.9
- Accelerates convergence, dampens oscillations

### 5.2.2 Nesterov Accelerated Gradient (NAG)

$$v_t = \beta v_{t-1} + \eta \nabla_\theta \mathcal{L}(\theta_t - \beta v_{t-1})$$
$$\theta_{t+1} = \theta_t - v_t$$

- Computes gradient at approximate future position
- Optimal $O(1/t^2)$ convergence rate for convex functions
- Better at preventing overshooting

## 5.3 Adaptive Optimizers

### 5.3.1 AdaGrad (Adaptive Gradient)

$$G_t = G_{t-1} + (\nabla_\theta \mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

- Per-parameter learning rates
- Excellent for sparse data (embeddings, categorical features)
- Problem: Learning rates vanish over time

### 5.3.2 RMSprop (Root Mean Square Propagation)

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)(\nabla_\theta \mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

- Uses moving average instead of cumulative sum
- Fixes AdaGrad's vanishing learning rate problem
- Good for RNNs and non-stationary problems

### 5.3.3 Adam (Adaptive Moment Estimation)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L}(\theta_t)$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L}(\theta_t))^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

- Combines momentum (first moment) and RMSprop (second moment)
- Bias correction ensures good performance from start
- Most widely used optimizer in deep learning
- Default parameters: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

### 5.3.4 AdamW (Adam with Weight Decay)

$$\theta_{t+1} = \theta_t - \eta \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

- Decouples weight decay from gradient-based updates
- Better generalization, especially for Transformers and LLMs
- Default for BERT, GPT, Vision Transformers

### 5.3.5 Nadam (Nesterov Adam)

Combines Adam with Nesterov momentum, providing faster convergence for many problems.

## 5.4 Optimizers Comparison

**Table 8: Optimizers Summary**

| Optimizer | Year | Key Idea | Best For |
|---|---|---|---|
| SGD | 1951 | Basic gradient descent | Simple problems |
| Momentum | 1964 | Velocity accumulation | Deep networks, ravines |
| NAG | 1983 | Look-ahead gradient | Convex optimization |
| AdaGrad | 2011 | Per-parameter sum | Sparse data, embeddings |
| RMSprop | 2012 | Moving average | RNNs, non-stationary |
| Adam | 2014 | Momentum + RMSprop + bias correction | General purpose |
| AdamW | 2017 | Decoupled weight decay | Transformers, LLMs |
| Nadam | 2016 | Adam + Nesterov | Noisy gradients |

## 5.5 Practical Recommendations

- **Start with Adam** ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) for most problems
- **Use SGD with momentum** ($\eta = 0.01$, $\beta = 0.9$) for computer vision (CNNs)
- **Use AdamW** ($\eta = 0.001$, $\lambda = 0.01$) for Transformers and LLMs
- **Use RMSprop** ($\eta = 0.001$, $\beta = 0.9$) for RNNs and LSTMs
- **Use AdaGrad** ($\eta = 0.01$) for sparse data (word embeddings)

## 5.6 Learning Rate Selection Guide

- **Too high** ($\eta > 0.1$): Divergence, loss explodes to NaN
- **High** ($\eta \approx 0.01 - 0.1$): Fast convergence but may oscillate
- **Good** ($\eta \approx 0.001 - 0.01$): Stable convergence for most optimizers
- **Low** ($\eta \approx 0.0001 - 0.001$): Slow but stable convergence
- **Too low** ($\eta < 0.0001$): Very slow convergence, may stall

## 5.7 Learning Rate Scheduling Strategies

- **Step decay**: Reduce LR by factor (0.1) every few epochs
- **Exponential decay**: $\eta_t = \eta_0 \cdot e^{-kt}$
- **Cosine annealing**: Cyclical schedule for better exploration
- **Warmup**: Gradually increase LR from 0 to target over first few epochs
- **Reduce on plateau**: Reduce LR when validation loss stops improving

# 6 Batch Size and Epochs

## 6.1 Introduction to Hyperparameters

**Définition 6.1 : Hyperparameters**

Configuration variables set before the learning process begins that control the behavior of the training algorithm. Unlike model parameters (weights), hyperparameters are not learned from data but must be manually tuned.

**Propriété 6.1 : Categories of Hyperparameters**

- **Optimization-related**: Learning rate, batch size, momentum, number of epochs
- **Model-related**: Number of layers, number of neurons, activation functions
- **Regularization-related**: Dropout rate, weight decay
- **Data-related**: Train/validation/test split ratio, data augmentation parameters

## 6.2 Batch Size

**Définition 6.2 : Batch Size**

The number of training samples processed before the model's internal parameters are updated. It determines how many examples the model sees before computing the gradient and performing a weight update.

### 6.2.1 Mathematical Foundation

For a dataset of $N$ samples, with batch size $B$, the gradient estimate is:

$$\nabla \mathcal{L}_B = \frac{1}{B} \sum_{i=1}^{B} \nabla \mathcal{L}(x_i, y_i; \theta)$$

The true gradient (full batch) would be:

$$\nabla \mathcal{L}_{\text{true}} = \frac{1}{N} \sum_{i=1}^{N} \nabla \mathcal{L}(x_i, y_i; \theta)$$

The variance of the batch gradient estimate is inversely proportional to batch size:

$$\text{Var}(\nabla \mathcal{L}_B) = \frac{1}{B} \text{Var}(\nabla \mathcal{L}_{\text{single}})$$

### 6.2.2 Types of Batch Sizes

**Full Batch Gradient Descent** ($B = N$) : Exact gradient, deterministic updates, guaranteed convergence for convex functions, impractical for large datasets.

**Mini-Batch Gradient Descent** ($1 < B < N$) : Most common approach, balances stability and efficiency, typical values: 16, 32, 64, 128, 256.

**Stochastic Gradient Descent** ($B = 1$) : Maximum noise, fast per-iteration, excellent for online learning, requires careful learning rate scheduling.

### 6.2.3 Impact of Batch Size on Training

- **Gradient Variance**: Small batches: high variance, noisy updates; Large batches: low variance, stable updates
- **Convergence Speed**: Small batches: faster per iteration but more iterations needed; Large batches: slower per iteration but fewer iterations needed
- **Generalization**: Small batches often lead to better generalization (noise acts as implicit regularization)
- **Memory Usage**: Linear relationship: Memory $\propto B$
- **Parallelization Efficiency**: Larger batches leverage parallel computation better

### 6.2.4 Batch Size Categories

**Table 9: Batch Size Categories and Their Characteristics**

| Category | Advantages | Disadvantages |
|---|---|---|
| Very Small (1-4) | Max regularization, escapes sharp minima | Very noisy, unstable |
| Small (4-16) | Good generalization | Noisy gradients, requires tuning |
| Medium (16-64) | Balance of speed and stability | Moderate memory usage |
| Large (64-256) | Stable gradients, fast per epoch | May overfit, memory intensive |
| Very Large (256-1024) | Very stable, hardware efficient | Poor generalization |

### 6.2.5 The Linear Scaling Rule

**Théorème 6.1 : Linear Scaling Rule - Goyal et al. 2017**

When increasing the batch size by a factor of $k$, the learning rate should be increased by the same factor to maintain training dynamics:

$$\eta_{\text{new}} = \eta_{\text{old}} \cdot k$$

This rule holds for SGD with momentum and is approximately valid for Adam.

### 6.2.6 Gradient Accumulation

A technique to simulate larger batch sizes when memory is limited. Gradients are accumulated over several forward/backward passes before performing a weight update.

```
effective_batch_size = 256
actual_batch_size = 32
accumulation_steps = effective_batch_size // actual_batch_size

optimizer.zero_grad()
for i, (inputs, labels) in enumerate(dataloader):
    outputs = model(inputs)
    loss = criterion(outputs, labels) / accumulation_steps
    loss.backward()

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

## 6.3 Epochs

**Définition 6.3 : Epoch**

One complete pass of the entire training dataset through the learning algorithm. During an epoch, every sample in the training set has been used once to update the model parameters.

**Définition 6.4 : Iteration**

A single parameter update using one batch of data. The number of iterations per epoch depends on dataset size and batch size.

### 6.3.1 Mathematical Relationships

$$\text{Iterations per epoch} = \left\lceil \frac{N_{\text{train}}}{B} \right\rceil$$

$$\text{Total training steps} = E \times \left\lceil \frac{N_{\text{train}}}{B} \right\rceil$$

$$\text{Training time} = E \times \left\lceil \frac{N_{\text{train}}}{B} \right\rceil \times T_{\text{iteration}}$$

### 6.3.2 Impact of Number of Epochs

**Underfitting Phase (Too few epochs)** : Training loss still decreasing, validation loss still decreasing, model hasn't learned all patterns

**Optimal Phase (Right number of epochs)** : Minimum validation loss achieved, best generalization performance

**Overfitting Phase (Too many epochs)** : Training loss continues to decrease, validation loss starts increasing, model memorizes noise

### 6.3.3 Determining Optimal Epochs

**Définition 6.5 : Early Stopping**

A regularization technique that stops training when validation performance stops improving for a specified number of epochs (patience), preventing overfitting and saving computational resources.

**Table 10: Patience Guidelines by Dataset Size**

| Dataset Size | Architecture | Recommended Patience |
|---|---|---|
| Small ($< 10$k) | Simple MLP | 20-50 |
| Small ($< 10$k) | CNN | 30-100 |
| Medium (10k-100k) | CNN | 10-30 |
| Medium (10k-100k) | Transformer | 5-15 |
| Large (100k-1M) | CNN | 5-20 |
| Very Large ($> 1$M) | Any | 2-5 |

### 6.3.4 Epoch Guidelines by Architecture

Table 11: Typical Number of Epochs by Architecture

| Architecture | Dataset | Typical Epochs |
|---|---|---|
| MLP | MNIST (60k) | 20-50 |
| MLP | CIFAR-10 (50k) | 50-100 |
| CNN | MNIST | 10-20 |
| CNN | CIFAR-10/100 | 50-200 |
| CNN | ImageNet (1.3M) | 90-100 |
| ResNet | CIFAR-10 | 200-300 |
| ResNet | ImageNet | 90-100 |
| Transformer | Machine Translation | 30-50 |
| Transformer | Fine-tuning | 3-10 |
| GANs | CIFAR-10 | 200-500 |
| LSTM/RNN | Text classification | 20-50 |

## 6.4 Relationship Between Batch Size and Epochs

**Propriété 6.2 : Interdependence**

For a fixed dataset size, batch size and number of epochs determine the total number of weight updates:

$$\text{Total updates} = \text{epochs} \times \left\lceil \frac{N}{B} \right\rceil$$

To achieve the same number of updates when changing batch size:

$$\text{epochs}_2 = \text{epochs}_1 \times \frac{B_1}{B_2}$$

## 6.5 Practical Recommendations

- **Start with batch size 32** - good default for most architectures
- **Use powers of 2** (16, 32, 64, 128, 256) for GPU efficiency
- **Monitor validation loss** to determine optimal epochs
- **Apply early stopping** with patience = 5-20 epochs
- **Scale learning rate** with batch size using linear scaling rule
- **Use gradient accumulation** for effective larger batches when memory limited
- **Consider dataset size** - larger datasets typically need fewer epochs
- **For very deep networks**, start with smaller batches to avoid instability

# 7 Validation Techniques

## 7.1 Fundamental Concepts

**Définition 7.1 : Generalization**

The ability of a model to perform accurately on new, unseen data that was not used during training. A model that generalizes well has learned the underlying patterns rather than memorizing the training data.

**Définition 7.2 : Overfitting**

A modeling error that occurs when a function is too closely fit to a limited set of data points, capturing noise instead of the underlying relationship. Overfit models perform well on training data but poorly on unseen data.

**Définition 7.3 : Underfitting**

A modeling error that occurs when a model is too simple to capture the underlying structure of the data, performing poorly on both training and test data.

## 7.2 Bias-Variance Tradeoff

**Théorème 7.1 : Bias-Variance Decomposition**

For a regression model with squared error loss, the expected generalization error can be decomposed as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2$$

where $\text{Bias}[\hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - f(x)$, $\text{Var}[\hat{f}(x)] = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$, and $\sigma^2$ is irreducible noise.

## 7.3 Data Splitting

### 7.3.1 Train/Validation/Test Split

- **Training set (60-80%)**: Used for model learning and parameter updates
- **Validation set (10-20%)**: Used for hyperparameter tuning and model selection
- **Test set (10-20%)**: Used for final, one-time evaluation of generalization performance

## 7.4 Cross-Validation Techniques

### 7.4.1 k-Fold Cross-Validation

- Data is divided into $k$ equal-sized folds (typically $k = 5$ or 10)

- Model trained $k$ times, each time using $k-1$ folds for training and 1 fold for validation
- Final performance is average across all $k$ trials
- Provides more reliable estimate of model performance, especially for small datasets

### 7.4.2 Stratified k-Fold

Maintains the proportion of classes in each fold, essential for imbalanced classification problems.

### 7.4.3 Leave-One-Out Cross-Validation (LOOCV)

Special case of k-fold where $k = N$ (number of samples). Computationally expensive but useful for very small datasets.

## 7.5 Learning Curves

Learning curves plot training and validation performance as a function of training set size or number of epochs. They help diagnose:

- **High bias (underfitting)**: Both curves converge at high error, gap is small
- **High variance (overfitting)**: Large gap between training and validation curves
- **Optimal fit**: Small gap, both curves at low error

## 7.6 Performance Metrics

### 7.6.1 Classification Metrics

**Confusion Matrix:**

- **TP (True Positives)**: Correctly predicted positive class
- **TN (True Negatives)**: Correctly predicted negative class
- **FP (False Positives)**: Incorrectly predicted positive (Type I error)
- **FN (False Negatives)**: Incorrectly predicted negative (Type II error)

**Derived Metrics:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

**ROC Curve and AUC:**

- ROC curve plots True Positive Rate vs False Positive Rate at various thresholds
- AUC (Area Under the Curve) measures overall classifier performance
- AUC = 1.0: perfect classifier, AUC = 0.5: random classifier

### 7.6.2 Regression Metrics

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

## 7.7 Early Stopping Techniques

### 7.7.1 Fixed Epoch Stopping

Train for a predetermined number of epochs regardless of performance.

- **Advantages**: Simple, deterministic, predictable training time
- **Disadvantages**: May stop too early (underfitting) or too late (overfitting)

### 7.7.2 Naive Best Solution Stopping

Stop at first increase in validation loss.

- **Problem**: Validation loss often fluctuates; stopping at first increase may be premature

### 7.7.3 Early Stopping with Patience

Stop only after validation performance has not improved for a specified number of epochs (patience).

- **Patience = 5-10**: Common for small datasets
- **Patience = 10-20**: Typical for medium datasets
- **Patience = 20-50**: For large datasets with noisy validation

### 7.7.4 Early Stopping with Improvement Threshold

Requires minimum improvement ($\Delta_{min}$) to reset patience counter.

$$\text{Significant Improvement} = \frac{L_{\text{old}} - L_{\text{new}}}{L_{\text{old}}} > \Delta_{\text{min}}$$

## 7.8 Section Summary

1. Validation fundamentals: generalization, overfitting, underfitting, bias-variance trade-off
2. Data splitting: train/validation/test split (60-80/10-20/10-20%)
3. Cross-validation: k-fold, stratified k-fold, LOOCV
4. Early stopping: fixed epoch, naive, patience-based, threshold-based
5. Classification metrics: accuracy, precision, recall, F1-score, ROC-AUC
6. Regression metrics: MSE, RMSE, MAE, Rš

# 8 First TensorFlow Example

## 8.1 Introduction to TensorFlow/Keras

**Définition 8.1 : TensorFlow**

An open-source machine learning framework developed by Google that provides a comprehensive ecosystem for building, training, and deploying deep learning models.

**Définition 8.2 : Keras**

A high-level neural networks API, integrated as TensorFlow's official frontend, that enables fast experimentation with user-friendly interfaces.

## 8.2 MNIST Classification Example

**Définition 8.3 : MNIST Dataset**

A classic dataset of handwritten digits (0-9) containing 60,000 training images and 10,000 test images. Each image is 28Œ28 pixels in grayscale.

### 8.2.1 Data Loading and Preprocessing

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.
    load_data()

# Normalize pixel values to [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten images (28x28 = 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)
```

### 8.2.2 Model Architecture

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(784,)
        ,
                       name='hidden_layer_1'),
    keras.layers.Dense(64, activation='relu', name='
        hidden_layer_2'),
```

```
    keras.layers.Dense(10, activation='softmax', name='
        output_layer')
])

model.summary()
```

### 8.2.3 Model Compilation

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

### 8.2.4 Model Training

```
history = model.fit(
    x_train, y_train,
    batch_size=32,
    epochs=10,
    validation_split=0.2,
    verbose=1,
    shuffle=True
)
```

### 8.2.5 Model Evaluation

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")

predictions = model.predict(x_test[:10])
predicted_classes = np.argmax(predictions, axis=1)
```

## 8.3 Understanding Dense Layers

**Définition 8.4 : Dense Layer**

A densely connected (fully connected) layer where each neuron receives input from all neurons in the previous layer:

$$\mathbf{a}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

**Propriété 8.1 : Parameter Count**

The number of parameters in a dense layer is:

$$\text{Parameters} = (d_{l-1} \times d_l) + d_l$$

**Table 12: Parameter count in MNIST model**

| Layer | Input dim | Output dim | Parameters |
|---|---|---|---|
| Hidden Layer 1 | 784 | 128 | 100,480 |
| Hidden Layer 2 | 128 | 64 | 8,256 |
| Output Layer | 64 | 10 | 650 |
| **Total** | | | **109,386** |

## 8.4 Advanced Model Configurations

### 8.4.1 Adding Regularization

```
from tensorflow.keras import regularizers

model = keras.Sequential([
    keras.layers.Dense(128, activation='relu',
                       kernel_regularizer=regularizers.l2(0.001),
                       input_shape=(784,)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(64, activation='relu',
                       kernel_regularizer=regularizers.l2(0.001)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(10, activation='softmax')
])
```

### 8.4.2 Early Stopping Callback

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)

history = model.fit(
    x_train, y_train,
    epochs=50,
    validation_split=0.2,
    callbacks=[early_stopping]
```

```
)
```

## 8.5 Model Saving and Loading

```
# Save entire model
model.save('mnist_model.h5')

# Save only weights
model.save_weights('mnist_weights.h5')

# Load model
loaded_model = keras.models.load_model('mnist_model.h5')

# Load weights (requires identical architecture)
model.load_weights('mnist_weights.h5')
```

## 8.6 Practical Recommendations

- Always normalize input data to [0,1] or standardize to mean 0, variance 1
- Start with simple architectures and gradually increase complexity
- Use validation data to monitor for overfitting
- Implement early stopping to save training time
- Visualize training curves to diagnose learning problems
- Experiment with different optimizers (Adam is good default)
- Use batch sizes that are powers of 2 (32, 64, 128) for GPU efficiency

# 9 Regularization Techniques

## 9.1 Why Regularization?

**Définition 9.1 : Regularization**

Any technique that modifies the learning algorithm to improve generalization performance on unseen data, typically by reducing overfitting without significantly increasing bias.

## 9.2 The 8 Most Important Regularization Techniques

### 9.2.1 1. L2 Regularization (Weight Decay)

$$\mathcal{L}_{reg} = \mathcal{L}_{original} + \frac{\lambda}{2}\sum_i w_i^2$$

- **Parameter**: $\lambda$ (regularization strength), typically $[10^{-5}, 10^{-3}]$
- **Effect**: All weights are shrunk toward zero but not exactly zero
- **Gradient**: $\frac{\partial \mathcal{L}_{reg}}{\partial w_i} = \frac{\partial \mathcal{L}_{original}}{\partial w_i} + \lambda w_i$
- Also known as: Ridge regression, weight decay

```
from tensorflow.keras import regularizers
model.add(keras.layers.Dense(64, activation='relu',
          kernel_regularizer=regularizers.l2(0.001)))
```

### 9.2.2 2. L1 Regularization (Lasso)

$$\mathcal{L}_{reg} = \mathcal{L}_{original} + \lambda\sum_i |w_i|$$

- **Effect**: Some weights become exactly zero (feature selection)
- **Gradient**: $\frac{\partial \mathcal{L}_{reg}}{\partial w_i} = \frac{\partial \mathcal{L}_{original}}{\partial w_i} + \lambda \cdot \text{sign}(w_i)$
- Also known as: Lasso regression, sparse regularization

```
model.add(keras.layers.Dense(64, activation='relu',
          kernel_regularizer=regularizers.l1(0.001)))
```

### 9.2.3 3. Dropout

- Randomly drops (sets to zero) a proportion of neurons during training
- **During training**: Each neuron kept with probability $p$ (typically 0.2-0.5)
- **During inference**: All neurons active, weights scaled by $p$
- Prevents co-adaptation, acts as ensemble of $2^N$ networks

```
from tensorflow.keras.layers import Dropout
model.add(Dropout(0.5))  # 50% dropout
```

### 9.2.4 4. Batch Normalization

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$
$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$$

- Normalizes activations to zero mean and unit variance across the batch
- Learnable scale ($\gamma$) and shift ($\beta$) parameters
- Allows higher learning rates, reduces dependence on initialization
- Acts as a regularizer (can sometimes replace dropout)
- Works poorly with small batch sizes (<16)

```
from tensorflow.keras.layers import BatchNormalization
model.add(keras.layers.Dense(128, activation='relu'))
model.add(keras.layers.BatchNormalization())
```

### 9.2.5 5. Early Stopping

- Stops training when validation performance stops improving
- Prevents overfitting automatically
- Saves computational resources
- Requires validation set
- Hyperparameters: patience (5-20), min delta

```
from tensorflow.keras.callbacks import EarlyStopping
early_stopping = EarlyStopping(
    monitor='val_loss', patience=10, restore_best_weights=True)
```

### 9.2.6 6. Data Augmentation

- Creates modified versions of training data through random transformations
- Effectively increases dataset size
- Improves robustness and generalization

**Common augmentations by domain:**

- **Images**: Rotation, flipping, cropping, zoom, color jitter, noise
- **Text**: Synonym replacement, back-translation, word dropout
- **Audio**: Time stretching, pitch shifting, noise addition

```
from tensorflow.keras.layers import RandomFlip, RandomRotation
data_augmentation = keras.Sequential([
    RandomFlip("horizontal"),
    RandomRotation(0.1),
    RandomZoom(0.1)
])
```

### 9.2.7 7. Label Smoothing

$$y_{\text{smooth}} = (1 - \epsilon) \cdot y_{\text{hard}} + \frac{\epsilon}{K}$$

where $K$ is number of classes, $\epsilon$ is smoothing factor (typically 0.1).

- Replaces hard one-hot labels with soft targets
- Prevents overconfidence in predictions
- Improves model calibration
- Reduces effect of label noise

```
model.compile(
    loss=keras.losses.CategoricalCrossentropy(label_smoothing
       =0.1),
    optimizer='adam', metrics=['accuracy'])
```

### 9.2.8 8. Gradient Clipping

$$\text{if } \|\nabla\mathcal{L}\| > c: \quad \nabla\mathcal{L} \leftarrow \frac{c}{\|\nabla\mathcal{L}\|}\nabla\mathcal{L}$$

- Limits magnitude of gradients during backpropagation
- Prevents exploding gradients
- Essential for RNNs and very deep networks
- Typical clipping value $c \in [0.5, 5.0]$

```
from tensorflow.keras.optimizers import Adam
optimizer = Adam(learning_rate=0.001, clipnorm=1.0)
```

## 9.3 Regularization Comparison

**Table 13: Regularization Techniques Summary**

| Technique | How It Works | Typical Values |
|---|---|---|
| L2 | Penalizes squared weights | $\lambda \in [10^{-5}, 10^{-3}]$ |
| L1 | Penalizes absolute weights | $\lambda \in [10^{-5}, 10^{-4}]$ |
| Dropout | Randomly drops neurons | Rate $p \in [0.2, 0.5]$ |
| BatchNorm | Normalizes activations | momentum $= 0.99$ |
| Early Stopping | Stops when no improvement | patience $5 - 20$ |
| Data Augmentation | Creates modified data | Domain-specific |
| Label Smoothing | Softens labels | $\epsilon = 0.1$ |
| Gradient Clipping | Limits gradient magnitude | $c \in [0.5, 5.0]$ |

## 9.4 Practical Recommendations

- **Small dataset**: Use L2 + Dropout + Data Augmentation
- **Large dataset**: BatchNorm + Early Stopping usually sufficient
- **Deep networks**: BatchNorm is essential
- **CNNs**: Data Augmentation + BatchNorm + Dropout

- **RNNs/LSTMs**: Gradient Clipping + Dropout
- **Transformers**: LayerNorm + Dropout + Label Smoothing
- **High-dimensional sparse data**: L1 regularization

# 10 Model Saving and Deployment

## 10.1 Why Model Persistence Matters

- **Reproducibility**: Save exact model state for research and debugging
- **Deployment**: Export models for production use
- **Checkpointing**: Save intermediate states during long training
- **Sharing**: Distribute models to other researchers/developers
- **Versioning**: Maintain multiple model versions
- **Cost savings**: Avoid retraining models from scratch

## 10.2 Saving and Loading Models

### 10.2.1 What Gets Saved?

**Table 14: Components Saved in Different Formats**

| Component | Weights Only | H5 Model | SavedModel |
|---|---|---|---|
| Model architecture | No | Yes | Yes |
| Weights/parameters | Yes | Yes | Yes |
| Training configuration | No | Yes | Yes |
| Optimizer state | No | Yes | Yes |
| Loss function | No | Yes | Yes |
| Metrics | No | Yes | Yes |

### 10.2.2 Saving Weights Only

```
# Save weights
model.save_weights('model_weights.h5')

# Load weights (requires identical architecture)
model = keras.Sequential([...])  # Recreate architecture
model.load_weights('model_weights.h5')
```

### 10.2.3 Saving Complete Model (H5 Format)

```
# Save complete model
model.save('my_model.h5')

# Load complete model
loaded_model = keras.models.load_model('my_model.h5')
```

### 10.2.4 SavedModel Format (Recommended)

```
# Save in SavedModel format
model.save('my_model')  # Creates directory

# Load SavedModel
loaded_model = keras.models.load_model('my_model')
```

### 10.2.5 Checkpoints During Training

```
from tensorflow.keras.callbacks import ModelCheckpoint

# Save best model only
checkpoint = ModelCheckpoint(
    'best_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max'
)

history = model.fit(x_train, y_train, validation_split=0.2,
                    epochs=50, callbacks=[checkpoint])
```

## 10.3 Model Conversion and Optimization

### 10.3.1 TensorFlow Lite for Mobile/Edge

```
# Convert to TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save TFLite model
with open('model.tflite', 'wb') as f:
    f.write(tflite_model)
```

**Quantization options:**

- **Float16**: 2x size reduction, minimal accuracy loss
- **Dynamic range**: 3-4x reduction, small accuracy loss
- **INT8 (full)**: 4x reduction, small-medium accuracy loss

### 10.3.2 TensorFlow.js for Web Deployment

```
# Install: pip install tensorflowjs
import tensorflowjs as tfjs
tfjs.converters.save_keras_model(model, 'tfjs_model')
```

### 10.3.3 ONNX for Framework Interoperability

```
# Install: pip install tf2onnx
import tf2onnx
spec = (tf.TensorSpec((None, 784), tf.float32, name="input"),)
model_proto, _ = tf2onnx.convert.from_keras(model,
    input_signature=spec)
```

## 10.4 Production Deployment

### 10.4.1 TensorFlow Serving

```
# Save model with version number
model.save('models/1/')  # Version 1

# Run TensorFlow Serving with Docker
docker run -p 8501:8501 \
  --mount type=bind,source=$(pwd)/models,target=/models/my_model
     \
  -e MODEL_NAME=my_model -t tensorflow/serving

# Query with REST API
curl -d '{"instances": [[0.1, 0.2, ...]]}' \
  -X POST http://localhost:8501/v1/models/my_model:predict
```

### 10.4.2 REST API with Flask

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np

app = Flask(__name__)
model = tf.keras.models.load_model('my_model.h5')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    input_data = np.array(data['input'], dtype=np.float32)
    predictions = model.predict(input_data)
    return jsonify({'predictions': predictions.tolist()})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

### 10.4.3 FastAPI Alternative

```python
from fastapi import FastAPI
from pydantic import BaseModel
import tensorflow as tf
import numpy as np

app = FastAPI()
model = tf.keras.models.load_model('my_model.h5')

class PredictionRequest(BaseModel):
    input: list

@app.post("/predict")
async def predict(request: PredictionRequest):
    input_data = np.array(request.input, dtype=np.float32)
    predictions = model.predict(input_data)
    return {"predictions": predictions.tolist()}
```

## 10.5 Model Versioning and Monitoring

```python
# Save models with versioned paths
model.save(f'models/v1.0.0/')
model.save(f'models/v1.1.0/')

# Version metadata
metadata = {
    'v1.0.0': {'date': '2024-01-15', 'accuracy': 0.982},
    'v1.1.0': {'date': '2024-02-01', 'accuracy': 0.985}
}
```

## 10.6 Practical Recommendations

- **Development**: Use H5 format for simplicity
- **Production**: Use SavedModel format
- **Mobile/Edge**: Convert to TFLite with quantization
- **Web**: Convert to TensorFlow.js
- **Versioning**: Use semantic versioning (v1.0.0)
- **Monitoring**: Track latency, throughput, and predictions
- **Testing**: Always test loaded models
- **Checkpoints**: Save regularly during training

# General Conclusion

## Synthesis of Fundamental Concepts

Throughout this comprehensive journey into the world of Deep Learning, we have traversed a path that leads from the most abstract mathematical foundations to the most concrete industrial applications, from the first McCulloch-Pitts neurons to the transformer architectures that shape the current state of the art. This conclusion aims to provide a complete synthesis of the knowledge acquired, a reflection on contemporary challenges, and an opening towards the future horizons of this constantly evolving discipline.

### The Theoretical Pillars

Deep Learning rests on solid mathematical foundations that we have explored throughout this course. **Backpropagation** constitutes the algorithmic heart of deep learning, allowing neural networks to adjust their parameters through an elegant application of the chain rule. **Activation functions** introduce the essential non-linearities that enable networks to approximate complex functions, while **loss functions** provide the compass that guides optimization toward meaningful solutions.

The **universal approximation theorem** reassures us of the theoretical power of neural networks, while the **hierarchical processing principle** explains their practical efficiency. We now understand that depth is not merely a quantitative characteristic but a qualitative transformation that enables the emergence of increasingly abstract representations.

### Optimization and Regularization

The **optimizers** we studiedfrom simple SGD to adaptive methods like Adam and AdamWrepresent the fruit of decades of research aimed at making learning faster, more stable, and more efficient. We have seen how **momentum** accelerates convergence, how **adaptive learning rates** simplify hyperparameter tuning, and how techniques like **gradient clipping** tame the instability of deep architectures.

**Regularization** has revealed itself as the art of balancing bias and variance. Through **L1/L2 regularization**, **dropout**, **batch normalization**, and **data augmentation**, we have acquired a complete arsenal to combat overfitting and improve generalization. **Early stopping** reminds us that sometimes the best decision is knowing when to stop.

### Practical Implementation

The transition from theory to practice was accomplished through **TensorFlow/Keras**, where we implemented our first models, learned to monitor training through learning curves, and mastered the art of hyperparameter tuning. We discovered that the **batch size** and **number of epochs** are not arbitrary choices but critical decisions that influence convergence speed, memory consumption, and final performance.

**Validation techniques**from simple train/validation/test splits to k-fold cross-validationgave us the tools to honestly evaluate our models and detect overfitting before it compromises

deployment. The **metrics** we studied (accuracy, precision, recall, F1-score, MSE, MAE, etc.) allow us to measure performance in a way that aligns with business objectives.

### From Development to Production

Finally, we crossed the bridge from development to production by exploring **model persistence** and **deployment strategies**. We learned to save models in various formats (H5, SavedModel, TFLite, ONNX), to version them properly, and to deploy them via REST APIs, mobile applications, or cloud services. This completes the cycle from idea to impact.

## Perspectives and Future Directions

### Beyond the Perceptron: Architectural Horizons

This course has laid the foundation for understanding the fundamental building blocks of deep learning. The concepts we have masteredbackpropagation, activation functions, optimization, regularizationare the common language that unifies all neural architectures. Armed with this knowledge, we are now ready to explore the rich landscape of specialized architectures that will be covered in subsequent chapters:

- **Feedforward Networks (FFN)**: The multilayer perceptron, which we have already studied in detail, serves as the foundation upon which all other architectures are built. Its principles of dense connections and non-linear transformations are universal.
- **Convolutional Neural Networks (CNN)**: These architectures revolutionize computer vision by exploiting the spatial structure of data. Through convolution operations and pooling layers, CNNs learn hierarchical featuresfrom edges to object parts to complete entitieswhile dramatically reducing the number of parameters compared to fully connected networks.
- **Recurrent Neural Networks (RNN)**: Designed for sequential data, RNNs and their variants (LSTM, GRU) maintain an internal state that captures temporal dependencies. They are essential for natural language processing, time series forecasting, speech recognition, and any task where order matters.
- **Transformers and Attention Mechanisms**: The architecture that has redefined the state of the art in NLP and beyond. By replacing recurrence with self-attention, transformers enable parallel processing of sequences and capture long-range dependencies with unprecedented efficiency. Models like BERT and GPT are built on this foundation.
- **Generative Models**: GANs (Generative Adversarial Networks) and VAEs (Variational Autoencoders) learn to generate new data that resembles the training distribution, opening doors to synthetic data generation, style transfer, and creative applications.

- **Graph Neural Networks (GNN)**: Extending deep learning to non-Euclidean domains, GNNs operate on graph-structured data, with applications in social networks, molecular chemistry, and recommendation systems.

## Final Remarks

This course has equipped you with the essential toolkit of a deep learning practitioner: the mathematical language to understand cutting-edge research, the practical skills to implement and deploy models, and the conceptual framework to design experiments and interpret results.

The journey of deep learning is far from over. Each year brings breakthroughs that were once thought impossible. Yet, beneath each innovation lies the same fundamental principles we have studiedgradients flowing backward through computational graphs, nonlinearities carving decision boundaries, and optimization algorithms seeking minima in high-dimensional spaces.

As you continue your journey into the specialized architectures of the following chapters, carry with you this solid foundation. Whether you are building convolutional networks for computer vision, recurrent networks for sequence modeling, or transformers for language understanding, the core concepts remain your faithful compass.

Remember the words that opened our journey:

*"The journey of a thousand models begins with a single neuron and a single backpropagation step."*

You have taken that step. The thousand models now await your exploration.

**Prof. Abdelouahab Moussaoui**
*Ferhat Abbas Sétif 1 University*
February 27, 2026

*"In the next chapters, we will explore the diverse landscape of neural architectures: Feedforward Networks, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Transformers, and beyond."*